# Topic Modeling of News Articles

A code-level walkthrough of the project: every script, every function, every file the pipeline produces, and the NLP ideas that tie them together.

### What this document covers

- **Section 1** — what the project does and the three-stage pipeline shape.
- **Section 2** — how a machine learning project works in general, and how this one maps onto that pattern.
- **Section 3** — the project's file layout, end to end.
- **Section 4** — the raw data and why it needs cleaning.
- **Sections 5–9** — *prepare_artifacts.py* read top to bottom: cleaning, bag-of-words, LDA training, scoring, and post-processing — with the real code and a line-by-line explanation of each block.
- **Section 10** — why the pipeline saves artifacts at all, and every single file inside `artifacts/`: what it is, who writes it, who reads it, and what's inside.
- **Section 11** — pyLDAvis, the interactive topic map.
- **Section 12** — *app.py* read top to bottom: caching, and all three Streamlit tabs.
- **Sections 13–14** — how to run it, and what to build next.

### Final model at a glance

| 8,914 | 20 | –13.11 | 0.557 |
|---|---|---|---|
| articles modeled | topics discovered | log perplexity | coherence (c_v) |

*Lower (more negative) perplexity and higher coherence both point to a better-fitting topic model — Section 8 explains what these numbers actually measure and why coherence is the one to trust more.*

## 1. What this project does

This project takes a large collection of scraped news articles and automatically discovers the hidden “topics” running through them — without any human labeling the articles first. It is an example of **unsupervised learning**: the model is never told what the topics are, only shown the raw text, and it groups words and documents together based on how often they co-occur.

The end-to-end pipeline has three stages, and — importantly — they live in **two separate, independently-runnable Python scripts** rather than one:

- **Collect & clean** — raw article text is normalized so the model sees meaningful words instead of noise. Lives in `prepare_artifacts.py`.
- **Model** — a Latent Dirichlet Allocation (LDA) model is trained on the cleaned text to find 20 latent topics, then scored and post-processed. Also in `prepare_artifacts.py`; its output is a folder of files called *artifacts*.
- **Explore** — a Streamlit web app (`app.py`) loads those pre-computed artifacts and lets anyone browse the discovered topics interactively, instead of re-reading a static notebook.

This design mirrors how real machine-learning systems are built: a slow, offline **training** job and a fast, online **serving** layer that never re-trains. Section 2 walks through that general pattern, and Section 3 maps every file in this project

onto it.

## 2. How a machine learning project works, end to end

Before diving into this project's specific code, it helps to see the shape it's built on. Almost every machine learning project — whether it's a spam filter, a recommendation engine, or this topic model — moves through the same sequence of stages, because each stage solves a problem the previous one creates. This project is a small, complete, working example of that full sequence, from a raw CSV file to a person clicking around a web page.

### 2.1 The pipeline, stage by stage

### 2.2 The same seven stages, in general ML terms

| Generic stage | What it means | In this project |
|---|---|---|
| Ingest raw data | Collect or receive the raw dataset in whatever form it arrives. | final_data_topic_modelling.csv (Section 4) |
| Clean & preprocess | Normalize the raw data into a consistent form, free of noise that would confuse the model. | `clean_to_tokens()` (Section 5) |

| Generic stage | What it means | In this project |
|---|---|---|
| Represent as features | Turn cleaned data into the numeric form the chosen algorithm can actually consume. | Dictionary + bag-of-words corpus (Section 6) |
| Train | Fit a model's internal parameters to the features. | `gensim.models.LdaModel(...)` (Section 7) |
| Evaluate | Score the trained model with objective metrics, since — especially for unsupervised learning — there's rarely a ground truth to check against. | perplexity + coherence (Section 8) |
| Package as artifacts | Persist the trained model and every derived result so nothing downstream ever has to recompute them. | the `artifacts/` folder (Section 10) |
| Serve / infer | A separate, lightweight application loads the artifacts and answers real requests, fast. | `app.py` (Section 12) |
| Monitor & iterate | Watch how the model performs on real usage, and retrain when the data or the world drifts. | *not built in this practice project* — see Section 14 |

### 2.3 Why training and serving are split into two programs

The first five stages above are slow, run rarely, and are allowed to be messy — together they form `prepare_artifacts.py`'s job. The last two are fast, run constantly, and need to be reliable — that's `app.py`'s entire job, and nothing else. Splitting them brings a few concrete benefits that matter well beyond this practice project:

- **Cost** — training over ~9,000 articles takes a few minutes; re-running that for every single click in the app would make it unusable.
- **Dependencies** — training needs NLTK's downloaded corpora, the full corpus in memory, and CPU time for the training loop. Serving only needs to load a handful of small files and draw charts. Keeping them separate keeps the deployed app's footprint small and its behavior predictable.
- **Reliability** — a bug or a slow step in training can't take the live app down, because the app never calls into that code path at all.
- **Reusability** — the same artifacts could be loaded by a completely different consumer later (a REST API, a notebook, a second app) without repeating any of the expensive work.

This is exactly what the discipline of **MLOps** is largely about at production scale: versioned datasets, repeatable training pipelines, a registry of trained models, and a serving layer that only ever consumes finished artifacts. `prepare_artifacts.py` → `artifacts/` → `app.py` is a small, file-based version of that same pattern — Section 10 explains the middle piece, the artifacts themselves, in depth.

## 3. Project layout

| Path | Role |
|---|---|
| Topic_Modelling.ipynb | The original exploratory notebook. Where the pipeline was designed and tried out step by step, cell by cell, before being turned into a repeatable script. |
| final_data_topic_modelling.csv | Raw input: scraped news articles (Title, Content, ...). ~74 MB, latin1-encoded. |
| prepare_artifacts.py | The **training** script — run once (or whenever the data/pipeline changes). Cleans the text, trains the LDA model, scores it, and writes every file the app needs into `artifacts/`. |

| Path | Role |
|---|---|
| artifacts/ | The **output** of the training script and the only thing the app reads. 10 files, covered file-by-file in Section 10. |
| app.py | The **serving** layer — a Streamlit web app. Never trains anything; only loads artifacts and renders them. Starts in ~1–2 seconds. |
| requirements.txt | Python dependencies: streamlit, pandas, numpy, gensim, nltk, contractions, plotly, pyLDAvis, scipy. |
| run_app.bat | One-click launcher — activates the local `venv` and runs `streamlit run app.py`. |
| venv/ | The project's isolated Python environment (not part of the pipeline logic itself). |

## 4. The raw data

`final_data_topic_modelling.csv` holds one row per scraped news article. The pipeline only keeps two columns from it:

- **Title** — the headline, used later purely for display (it is never fed into the model).
- **Content** — the article body, the actual text the topic model learns from.

The file is read with `encoding="latin1"` because that's how it was scraped/saved; this occasionally mis-decodes UTF-8 punctuation (curly quotes, em-dashes) into garbled sequences like â€™ — a problem commonly called **mojibake**. A small helper repairs titles for display purposes:

```
def fix_mojibake(text: str) -> str:
    """Recover UTF-8 text that was mis-decoded as Latin-1 (e.g. curly quotes -> 'â€™')."""
    try:
        return text.encode("latin1").decode("utf-8")
    except (UnicodeDecodeError, UnicodeEncodeError):
        return text
```

*It works by reversing the mistake: re-encoding the already-broken string back into raw bytes as Latin-1, then decoding those bytes correctly as UTF-8. If that round-trip isn't valid (the text wasn't actually mis-decoded), it just falls back to the original string unchanged.*

Before any text cleaning happens, rows are also filtered for basic sanity:

```
raw = pd.read_csv(DATA_PATH, encoding="latin1")
df = raw[["Title", "Content"]].dropna().drop_duplicates().reset_index(drop=True)
df["Title"] = df["Title"].apply(fix_mojibake)
```

`dropna()` removes rows missing a title or body; `drop_duplicates()` removes exact repeat articles (common in scraped news data, where the same wire story gets syndicated). This leaves **8,933** articles heading into the cleaning stage.

## 5. The data cleaning pipeline

Raw news text is messy: contractions, punctuation, numbers, mixed capitalization, and common “filler” words that carry no topical meaning. All of the cleaning logic lives in one function, applied to every article's `Content`:

```
def clean_to_tokens(text: str) -> list[str]:
    text = " ".join(contractions.fix(w) for w in str(text).split())
    text = text.lower()
    text = re.sub(r"[^\w\s]", "", text)
    tokens = word_tokenize(text)
    tokens = [t for t in tokens if not any(c.isdigit() for c in t)]
    tokens = [t for t in tokens if t not in STOP_WORDS and len(t) > 2]
    tokens = [LEMMATIZER.lemmatize(t) for t in tokens]
    return tokens
```

Read top to bottom, each line strips away one specific kind of noise:

| Line | What it does | Why it matters |
|---|---|---|
| `contractions.fix(w)` | Expands “don't” → “do not”, word by word. | Keeps word forms consistent so “do” and “not” get counted the normal way, instead of the model treating “don't” as a single rare token. |
| `text.lower()` | “Election” → “election”. | Stops the model from treating the same word as two different vocabulary entries just because of capitalization. |
| `re.sub(r"[^\w\s]", "", text)` | Strips every character that isn't a letter, digit, or whitespace. | Punctuation carries no topical signal on its own; `\w` is the regex shorthand for “word character”, `\s` for whitespace. |
| `word_tokenize(text)` | Splits the cleaned string into a list of individual word tokens (NLTK's tokenizer, handles edge cases better than plain `.split()`). | LDA works on discrete tokens, not raw strings. |
| drop tokens with a digit | Removes any token containing a numeral, e.g. “nov23”. | Article-specific numbers (dates, scores, prices) are almost never shared across articles on the same topic, so they only add noise. |
| drop stopwords / short tokens | Removes common function words (“the”, “is”, “and”, ...) and anything ≤2 characters. | Stopwords appear in every document regardless of topic and dilute the signal; very short leftover tokens are usually junk from the regex step. |
| `LEMMATIZER.lemmatize(t)` | “running”, “ran” → “run”. | Merges different inflections of the same word into one shared token, so the model isn't splitting one concept's evidence across several near-duplicate tokens. |

`STOP_WORDS` and `LEMMATIZER` are built once at module load time (not inside the function, so they aren't rebuilt for every one of the ~9,000 articles):

```
for pkg in ("stopwords", "punkt", "punkt_tab", "wordnet"):
    nltk.download(pkg, quiet=True)

STOP_WORDS = set(stopwords.words("english"))
LEMMATIZER = WordNetLemmatizer()
```

*The four NLTK downloads are the pretrained resources the cleaning step depends on: the English stopword list, two tokenizer models (punkt / punkt_tab), and WordNet (the lexical database the lemmatizer looks words up in). They're cached locally after the first run, so this is effectively a no-op on subsequent runs.*

Applying the whole function to every article, then filtering out documents that came out nearly empty (fewer than 6 usable tokens — too little signal for LDA to use):

```
df["tokens"] = df["Content"].apply(clean_to_tokens)
df["word_count"] = df["tokens"].apply(len)
df = df[df["word_count"] > 5].reset_index(drop=True)
```

This is the slow step of the whole pipeline (tokenizing and lemmatizing ~9,000 articles one word at a time) and is exactly why it only runs inside `prepare_artifacts.py`, never inside the app. After this filter, **8,914** articles remain — the number quoted throughout this document.

# 6. Turning text into numbers: Bag-of-Words

Machine learning models work with numbers, not words, so the cleaned token lists have to become a numeric representation before LDA can use them. This project uses the classic **bag-of-words** approach via the Gensim library, in two steps.

### 6.1 Dictionary — word → integer ID

```
id2word = corpora.Dictionary(df["tokens"])
id2word.filter_extremes(no_below=5, no_above=0.5)
```

`corpora.Dictionary` scans every article's token list and assigns each distinct word a unique integer ID, building a vocabulary from the whole corpus. `filter_extremes` then prunes that vocabulary at both ends: `no_below=5` drops words appearing in fewer than 5 articles (too rare to generalize into a topic), and `no_above=0.5` drops words appearing in more than half of all articles (too common to distinguish one topic from another — effectively corpus-specific stopwords).

### 6.2 Corpus — each article as (word ID, count) pairs

```
corpus = [id2word.doc2bow(tokens) for tokens in df["tokens"]]
```

`doc2bow` (“document to bag-of-words”) converts one article's token list into a list of `(word_id, count)` tuples — e.g. an article that says “election” 4 times and “vote” twice becomes entries like `(id_of("election"), 4)`, `(id_of("vote"), 2)`. This throws away word order and grammar entirely; only word frequency matters, which is a deliberate simplification that keeps the model tractable. It's the same idea behind a search engine's inverted index, or a spam filter's word-frequency table.

# 7. Training the model: Latent Dirichlet Allocation (LDA)

LDA is a probabilistic model built on one core assumption: **every document is a mixture of topics, and every topic is a distribution over words.** A sports article might be 80% “football” topic and 20% “business” topic (e.g. a story about a club's finances); the “football” topic itself is really just a probability distribution that puts high weight on words like *match*, *goal*, and *team*.

LDA never sees topic labels — it only sees which words tend to appear together across thousands of documents, and works backward (via variational Bayesian inference) to infer both things above at once: what the topics are (as word distributions) and how much of each topic is in each document. The entire model is trained with one call:

```
lda_model = gensim.models.LdaModel(
    corpus=corpus,
    id2word=id2word,
    num_topics=NUM_TOPICS,
    random_state=100,
    update_every=1,
    chunksize=100,
    passes=10,
    alpha="auto",
    per_word_topics=True,
)
```

| Parameter | Value | Meaning |
|---|---|---|
| num_topics | 20 | How many latent topics to search for. Fixed in this project (`NUM_TOPICS = 20`); a natural next experiment (Section 14) is sweeping this value against coherence. |
| passes | 10 | How many full passes the training algorithm makes over the entire corpus. More passes let the model converge to a more stable set of topics, at the cost of training time. |
| chunksize | 100 | How many documents are processed together per mini-batch during each pass — a performance/memory knob, doesn't change what the model converges to. |
| update_every | 1 | Model parameters are updated after every chunk (online learning) rather than only once per full pass. |
| alpha | "auto" | Controls how many topics a typical document is expected to mix together (the document-topic Dirichlet prior). "auto" lets Gensim learn this from the data instead of fixing it upfront. |
| random_state | 100 | A fixed seed so re-running training reproduces the same topics, instead of a new random result every time. |
| per_word_topics | True | Also computes, for each word in each document, which topic(s) it was most likely assigned to — not used by this project's post-processing, but enables deeper per-word inspection later if needed. |

## 8. Scoring the model: perplexity & coherence

Because LDA is unsupervised, there's no ground-truth answer to check the topics against, so two standard statistical metrics are computed instead, right after training:

```
perplexity = lda_model.log_perplexity(corpus)
coherence_model = CoherenceModel(
    model=lda_model, texts=df["tokens"], dictionary=id2word, coherence="c_v"
)
with np.errstate(invalid="ignore"):
    coherence = coherence_model.get_coherence()
```

### Perplexity (this model: −13.11)

`log_perplexity` measures how well the trained model predicts the actual words in the (training) corpus, reported as a log-likelihood. Less negative (closer to zero) is technically “better fit”, but in practice perplexity is known to correlate poorly with how coherent the topics actually look to a human reader — which is exactly why the second metric matters more here.

### Coherence, c_v measure (this model: 0.557)

Coherence directly measures whether the top words within a single topic tend to actually co-occur together in real documents — a much better proxy for “would a human agree this is a real topic?”. The `np.errstate(invalid="ignore")` wrapper simply silences harmless NumPy warnings that the c_v calculation's internal cosine-similarity math can trigger on edge cases (e.g. a zero-vector comparison) — it doesn't change the computed score. Scores for this style of model typically range from about 0.3 (weak, mostly noise) to the low 0.6s (strong, well-separated topics), so 0.557 indicates a solidly coherent set of topics.

# 9. Post-processing: from model to browsable results

A trained LDA model on its own is just probability tables. The rest of `prepare_artifacts.py` turns it into the concrete, human-readable pieces the app actually displays.

### 9.1 Assigning a dominant topic to every article

```
dominant_topic, dominant_prob = [], []
for bow in corpus:
    dist = lda_model.get_document_topics(bow, minimum_probability=0.0)
    top_tid, top_p = max(dist, key=lambda x: x[1])
    dominant_topic.append(top_tid)
    dominant_prob.append(float(top_p))
df["dominant_topic"] = dominant_topic
df["topic_prob"] = dominant_prob
```

For every article, `get_document_topics` returns its full mixture across all 20 topics (e.g. 74% topic 2, 12% topic 7, ... down to near-zero for the rest); `minimum_probability=0.0` forces it to return every topic instead of silently dropping tiny ones. `max(..., key=lambda x: x[1])` then just picks the single highest-probability topic as that article's *dominant topic* — the label used everywhere else in the app.

### 9.2 Top words per topic and a short human-readable label

```
topic_words = {}
for tid in range(NUM_TOPICS):
    words = lda_model.show_topic(tid, topn=TOP_N_WORDS)
    topic_words[str(tid)] = [{"word": w, "weight": float(wt)} for w, wt in words]

topic_labels = {
    str(tid): " / ".join(w["word"] for w in topic_words[str(tid)][:3])
    for tid in range(NUM_TOPICS)
}
```

`show_topic(tid, topn=15)` returns that topic's 15 highest-probability words with their weights (each topic's *full* distribution spans the entire ~vocabulary, so this is just the most informative slice). `topic_labels` then stitches together just the top 3 words of each topic (e.g. `"israel / hamas / gaza"`) as a quick stand-in for “Topic 3” in the UI — it's derived data, not something the model computes on its own.

### 9.3 Representative headlines per topic

```
representative = {}
for tid in range(NUM_TOPICS):
    sub = df[df["dominant_topic"] == tid].sort_values("topic_prob", ascending=False)
    representative[str(tid)] = sub[["Title", "topic_prob"]]\
        .head(TOP_N_DOCS_PER_TOPIC).to_dict("records")
```

For each topic, this filters to the articles whose dominant topic is that one, sorts them by how strongly they match it, and keeps the top 8 — the app's “most representative headlines” list. A real example from this run, topic 2 (*israel / hamas / gaza*):

```
{
  "word": "israel",  "weight": 0.0735
  "word": "hamas",   "weight": 0.0554
  "word": "gaza",    "weight": 0.0391
  "word": "israeli", "weight": 0.0380
  "word": "attack",  "weight": 0.0308
  ...
}
# most representative headline:
# "'We are going to change the Middle East,' says Israeli premier"  (74.6% match)
```

### 9.4 Saving everything

Finally, the model, dictionary, per-document assignments, and every derived JSON file are written to `artifacts/`, and a pyLDAvis HTML map is rendered and saved alongside them. Section 10 explains why this step exists at all, then covers each output file in detail.

# 10. The artifacts folder: why it exists, and file by file

## 10.1 Why does the pipeline save artifacts at all?

An **artifact**, in machine learning engineering, is any durable output a pipeline stage produces and saves so that a later stage — or a later run, or a completely different program — can use it without redoing the work that created it. Section 2.3 introduced the general reason this project is split into a training program and a serving program; this is where that split becomes concrete. `prepare_artifacts.py` is the only code in this project allowed to be slow, and `artifacts/` is the checkpoint where everything it produced gets frozen and handed off.

Specifically, saving these files serves five distinct purposes:

- **Avoiding repeated cost** — training the model and cleaning ~9,000 articles takes real time (minutes, here). Without artifacts, that entire pipeline would have to re-run every time the app started, or worse, every time a user interacted with it.
- **Reproducibility** — `metrics.json` freezes the exact perplexity and coherence this specific run produced. Anyone can check exactly what a training run produced, months later, without re-running any code — the numbers quoted throughout this document came straight from that file.
- **Debuggability** — if a chart in the app looks wrong, the fastest way to investigate is to open the relevant artifact directly (read the JSON, unpickle the DataFrame) rather than re-running the full 10-stage pipeline to reproduce the bug.
- **Portability** — plain JSON, pickle, and HTML files can be zipped, copied, or deployed to a machine that has never seen `gensim`'s training code, the NLTK downloads, or the 74 MB raw CSV. It only needs enough code to *load* them.
- **Isolating dependencies** — training-only libraries and data (NLTK corpora, the full raw corpus in memory) never need to exist in the environment that serves the app; the app's dependency footprint stays small and predictable.

Put together, `artifacts/` is a small, file-based version of what a production ML system would call a **model registry**: a single, versionable place holding the one true output of training, which every downstream consumer treats as read-only.

## 10.2 The files, one by one

Everything `app.py` displays comes from these 10 files. Nothing in this folder is edited by hand — it is entirely regenerated by running `prepare_artifacts.py`.

**lda_model.gensim** — **Binary · gensim LdaModel**

**Produced by:** `lda_model.save(...)`
**Used by:** app.py → `gensim.models.LdaModel.load(...)` (cached with `@st.cache_resource`)
**Contents:** The trained model object itself — its hyperparameters (num_topics, alpha, etc.) and the machinery to compute a document's topic mixture on demand. This is the file Gensim's `SaveLoad` mechanism treats as the "main" pickle; the four files below are its companions, auto-split off because they're either large numpy arrays or logically separate sub-objects.

**lda_model.gensim.expElogbeta.npy** — **Binary · NumPy array (~2.1 MB)**

**Produced by:** gensim, automatically, as part of `lda_model.save(...)`
**Used by:** Loaded automatically whenever `lda_model.gensim` is loaded — the app never references it directly.
**Contents:** A num_topics × vocabulary-size matrix holding exp(E[log β]), the expected log topic-word distribution used internally during inference. Gensim's save mechanism moves any numpy array above a size threshold into its own `.npy` sidecar file rather than embedding it in the main pickle, which is why this one file alone is several megabytes.

### lda_model.gensim.state

**Binary · gensim LdaState**

**Produced by:** gensim, automatically, as part of `lda_model.save(...)`
**Used by:** Loaded automatically with the model; not read directly by app.py.
**Contents:** The variational-inference sufficient statistics accumulated during training (the “state” LDA needs to keep being trained further, or to recompute exact topic-word probabilities). Kept separate from the main model file because it's a distinct object in Gensim's class hierarchy, not because of size.

### lda_model.gensim.id2word

**Binary · pickled gensim Dictionary**

**Produced by:** gensim, automatically, as part of `lda_model.save(...)`
**Used by:** Loaded automatically with the model.
**Contents:** A copy of the dictionary the model was trained with, bound to the model object so it can turn topic word-IDs back into readable words on its own. This is separate from `dictionary.gensim` below (the standalone copy used to build the corpus) — the two are saved independently but contain the same vocabulary.

### dictionary.gensim

**Binary · gensim Dictionary (~852 KB)**

**Produced by:** `id2word.save(...)`
**Used by:** Not currently loaded by app.py (the model's own bundled copy is used instead); kept as a standalone artifact so the vocabulary can be reused independently — e.g. to encode a brand-new document into the same word-ID space without reloading the full model.
**Contents:** The word ↔ integer-ID mapping built in Section 6.1, after `filter_extremes` pruning. This is the vocabulary every topic's word list and every document's bag-of-words representation is expressed in terms of.

### doc_topics.pkl

**Binary · pandas DataFrame pickle (~993 KB, 8,914 rows)**

**Produced by:** `df[[...]].to_pickle(...)`
**Used by:** app.py → `pd.read_pickle(...)`, used by the Overview tab (topic size chart) and Topic Explorer tab (article counts per topic)
**Contents:** One row per surviving article with columns `Title`, `word_count`, `dominant_topic`, `topic_prob`. This is the per-document result table — everything Section 9.1 computed, saved for instant reuse.

### topic_words.json

**Text · JSON (~23 KB)**

**Produced by:** `json.dump(topic_words, ...)`
**Used by:** app.py → Topic Explorer tab, to draw the “top words” bar chart
**Contents:** `{"0": [{"word": "first", "weight": 0.0xx}, ...], "1": [...], ...}` — one entry per topic ID (as a string key, since JSON object keys are always strings), each holding its top 15 words and their probability weights, from Section 9.2.

### topic_labels.json

**Text · JSON (~679 B)**

**Produced by:** `json.dump(topic_labels, ...)`
**Used by:** app.py → every dropdown and chart label that shows a topic name
**Contents:** `{"2": "israel / hamas / gaza", "3": "game / player / team", ...}` — the short 3-word label per topic from Section 9.2. The smallest file in the folder, but the one that makes the whole app readable instead of showing bare topic numbers.

**representative_docs.json** — Text · JSON (~27 KB)

**Produced by:** `json.dump(representative, ...)`
**Used by:** app.py → Topic Explorer tab, the "most representative headlines" list
**Contents:** `{"2": [{"Title": "...", "topic_prob": 0.746}, ...], ...}` — per topic, the 8 real headlines that match it most strongly, from Section 9.3.

**metrics.json** — Text · JSON (119 B)

**Produced by:** `json.dump({...}, ...)`
**Used by:** app.py → the page caption and the three Overview metric tiles
**Contents:** `{"perplexity": -13.109..., "coherence": 0.5573..., "num_docs": 8914, "num_topics": 20}` — the four global scalars from Section 8, exactly as computed, with no rounding (the app formats them for display).

**pyldavis.html** — Text · standalone HTML/JS document (~186 KB)

**Produced by:** `pyLDAvis.save_html(vis, ...)`
**Used by:** app.py → Topic Map tab, embedded via `st.components.v1.html(...)`
**Contents:** A fully self-contained, D3.js-powered interactive report with all of its topic coordinates and word frequencies baked directly into the file. Unlike every other artifact, it doesn't depend on the model, dictionary, or any other file at load time — it's already a complete visualization you could open directly in a browser, or email, independent of this project.

# 11. Visualizing the topics: pyLDAvis

A table of 20 topics, each with a list of top words, is hard to make sense of at a glance. The map is generated with two lines at the end of `prepare_artifacts.py`:

```
vis = gensimvis.prepare(lda_model, corpus, id2word)
pyLDAvis.save_html(vis, str(ARTIFACTS_DIR / "pyldavis.html"))
```

`gensimvis.prepare` runs multidimensional scaling on the topic-word distributions to lay every topic out as a bubble on a 2-D plane: bubble size reflects how much of the corpus that topic covers, and topics using similar vocabulary are plotted closer together. Clicking a bubble shows the words that most define that topic, ranked by a tunable *relevance* score (not just raw probability). This view makes two things visible immediately that a word list can't: which topics are large versus niche, and which topics overlap or blur into each other — a sign the topic count might be too high or too low for this corpus.

# 12. The Streamlit app (app.py)

The original notebook is a one-time, top-to-bottom script — useful for building the model, but not for letting someone else explore the results. `app.py` turns the saved artifacts into an interactive tool. It never trains anything; every function below is either a fast loader or a display block.

### 12.1 Loading artifacts, with caching

```
@st.cache_resource
def load_model():
    return gensim.models.LdaModel.load(str(ARTIFACTS_DIR / "lda_model.gensim"))

@st.cache_data
def load_json(name: str):
    with open(ARTIFACTS_DIR / name, encoding="utf-8") as f:
        return json.load(f)

@st.cache_data
def load_doc_topics():
    return pd.read_pickle(ARTIFACTS_DIR / "doc_topics.pkl")
```

Streamlit re-runs the entire script top-to-bottom on every user interaction (every click, every dropdown change), so without caching the app would reload the model and every JSON file from disk on every single click. The two decorators solve this differently: `@st.cache_resource` is for objects that shouldn't be copied or serialized (like the Gensim model, a heavy, stateful Python object) — it's kept as one shared instance in memory. `@st.cache_data` is for plain, serializable data (dicts, DataFrames) — Streamlit hashes the function's inputs and safely returns cached copies. Both only re-run the underlying function once per process, no matter how many times the script re-executes.

### 12.2 Guard clause and setup

```
if not ARTIFACTS_DIR.exists():
    st.error(
        "No artifacts found. Run `venv\\Scripts\\python.exe prepare_artifacts.py` "
        "first to generate the topic model."
    )
    st.stop()

topic_words = load_json("topic_words.json")
topic_labels = load_json("topic_labels.json")
representative_docs = load_json("representative_docs.json")
metrics = load_json("metrics.json")
doc_topics = load_doc_topics()

topic_ids = sorted(topic_words.keys(), key=int)
topic_display = {tid: f"Topic {int(tid) + 1}: {topic_labels[tid]}" for tid in topic_ids}
```

Before doing anything else, the app checks the artifacts folder actually exists and fails with a clear instruction rather than a confusing stack trace if `prepare_artifacts.py` hasn't been run yet. `topic_ids` sorts the string keys numerically (`key=int`) since plain string sort would put “10” before “2”. `topic_display` builds the final human-facing label for every topic once, up front, combining a 1-indexed topic number with its 3-word label from `topic_labels.json` — e.g. `"Topic 3: israel / hamas / gaza"`.

### 12.3 Overview tab

```
counts = doc_topics["dominant_topic"].value_counts().sort_index()
size_df = pd.DataFrame({
    "Topic": [topic_display[str(t)] for t in counts.index],
    "Articles": counts.values,
}).sort_values("Articles", ascending=True)
fig = px.bar(size_df, x="Articles", y="Topic", orientation="h", height=650)
st.plotly_chart(fig, use_container_width=True)
```

`value_counts()` on the `dominant_topic` column (loaded straight from `doc_topics.pkl`) gives the article count per topic in one call; sorting ascending before plotting a horizontal bar chart puts the largest topic at the top of the page, which is the natural reading order. Three `st.metric` tiles below it surface `metrics.json`'s article count, topic count, and coherence score directly.

### 12.4 Topic Explorer tab

```
selected_label = st.selectbox(
    "Choose a topic", options=topic_ids, format_func=lambda t: topic_display[t]
)
...
words_df = pd.DataFrame(topic_words[selected_label])
words_df = words_df.sort_values("weight", ascending=True)
fig_words = px.bar(words_df, x="weight", y="word", orientation="h", height=450)
...
docs = representative_docs[selected_label]
for d in docs:
    st.markdown(f"- {d['Title']}  \n  match: {d['topic_prob']:.0%}")
n_in_topic = int((doc_topics["dominant_topic"] == int(selected_label)).sum())
```

The dropdown stores the raw topic **ID** (a string like `"2"`) as its value but displays the friendly `topic_display` label via `format_func` — so the rest of the code can keep indexing the JSON dicts by that same ID. Selecting a topic drives two independent lookups straight into the pre-computed JSON: its top-word weights become a bar chart, and its representative headlines become a bulleted list with each one's match percentage. The article count at the bottom re-derives “how many articles have this as their dominant topic” on the fly from `doc_topics`, since that wasn't pre-aggregated anywhere.

### 12.5 Topic Map tab

```
pyldavis_path = ARTIFACTS_DIR / "pyldavis.html"
if pyldavis_path.exists():
    html = pyldavis_path.read_text(encoding="utf-8")
    st.components.v1.html(html, height=800, scrolling=True)
```

The simplest tab in the app: it just reads the pre-rendered pyLDAvis HTML file as raw text and hands it to Streamlit's `components.v1.html`, which embeds it as a sandboxed iframe. No parsing, no re-computation — the interactivity (hover, click, adjust relevance metric λ) is entirely self-contained JavaScript already baked into the file, exactly as described in Section 10.

## 13. Running it

From the *practice topic modeling* folder:

- **Regenerate the model** (only needed after changing the data or the pipeline): `venv\Scripts\python.exe prepare_artifacts.py`
- **Launch the app**: double-click `run_app.bat`, or run `venv\Scripts\streamlit run app.py`

## 14. Possible next steps

- **Modern embeddings (BERTopic)** — replace or complement bag-of-words LDA with sentence-embedding based topic modeling, which typically produces cleaner topics on short/noisy text without hand-picking the topic count.
- **Justify the topic count** — sweep `num_topics` against the coherence score and pick the value where coherence peaks, instead of using a fixed 20.
- **Temporal trends** — if publish dates can be recovered from the original source, plot how each topic's share of the news cycle rises and falls over time.
- **LLM-generated topic labels** — replace the current 3-word label with a short human-readable name generated from each topic's top words and example headlines.

- **The missing eighth stage** — add monitoring: track which topics real users click into most, and treat that as a signal for when the model needs retraining, closing the loop introduced in Section 2.

---

*Generated as detailed code & artifact documentation for the news-article topic modeling project and its Streamlit explorer.*